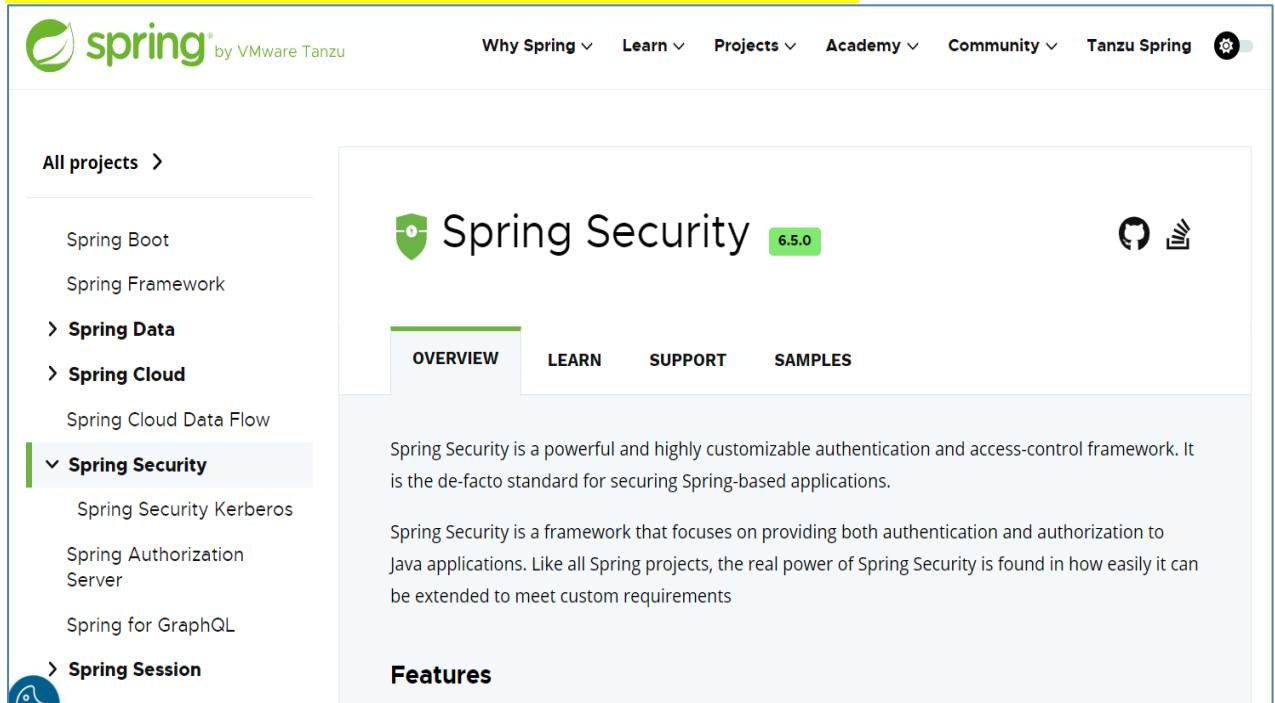
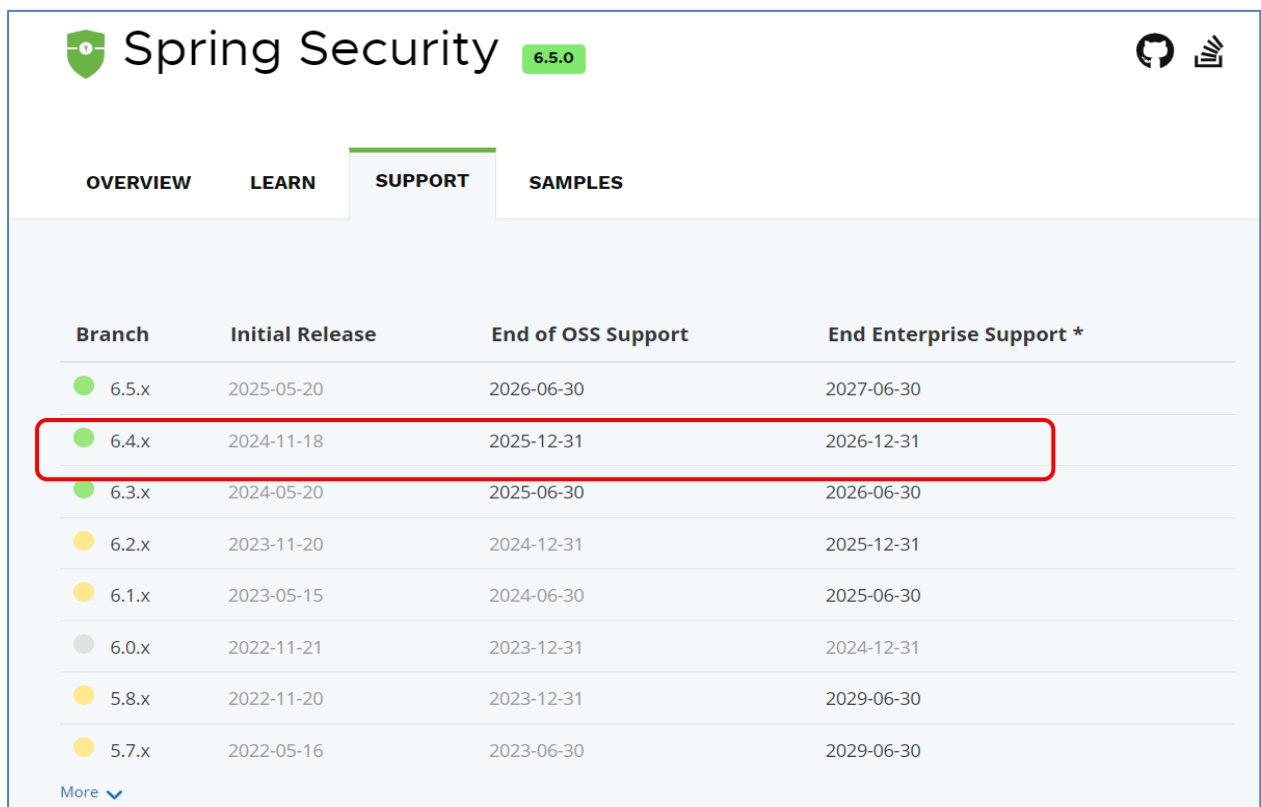


[Spring Security 웹 페이지]

<https://spring.io/projects/spring-security#overview>


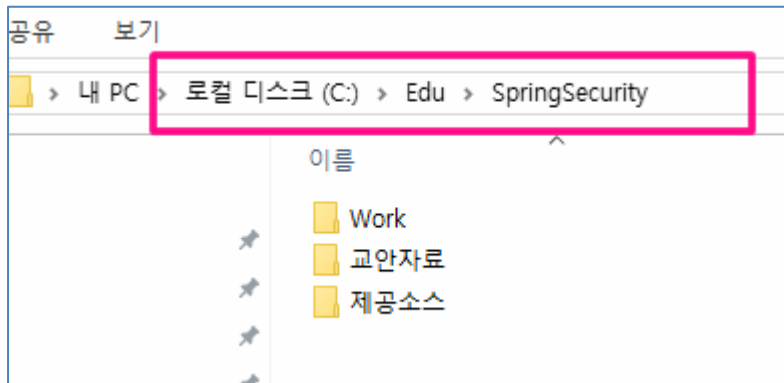
The screenshot shows the Spring Security overview page. The left sidebar lists various Spring projects, with 'Spring Security' highlighted. The main content area features the 'Spring Security 6.5.0' header, navigation tabs for 'OVERVIEW', 'LEARN', 'SUPPORT', and 'SAMPLES', and a brief description of the framework. The 'Features' section is partially visible at the bottom.



The screenshot shows the 'SUPPORT' tab of the Spring Security support matrix. The table lists various branches and their corresponding support dates. The row for '6.4.x' is highlighted with a red rectangle.

Branch	Initial Release	End of OSS Support	End Enterprise Support *
6.5.x	2025-05-20	2026-06-30	2027-06-30
6.4.x	2024-11-18	2025-12-31	2026-12-31
6.3.x	2024-05-20	2025-06-30	2026-06-30
6.2.x	2023-11-20	2024-12-31	2025-12-31
6.1.x	2023-05-15	2024-06-30	2025-06-30
6.0.x	2022-11-21	2023-12-31	2024-12-31
5.8.x	2022-11-20	2023-12-31	2029-06-30
5.7.x	2022-05-16	2023-06-30	2029-06-30

[학습용 폴더 생성]



Spring Security 기본동작을 알아보기 위한 첫 번째 실습

프로젝트 환경세팅 구성

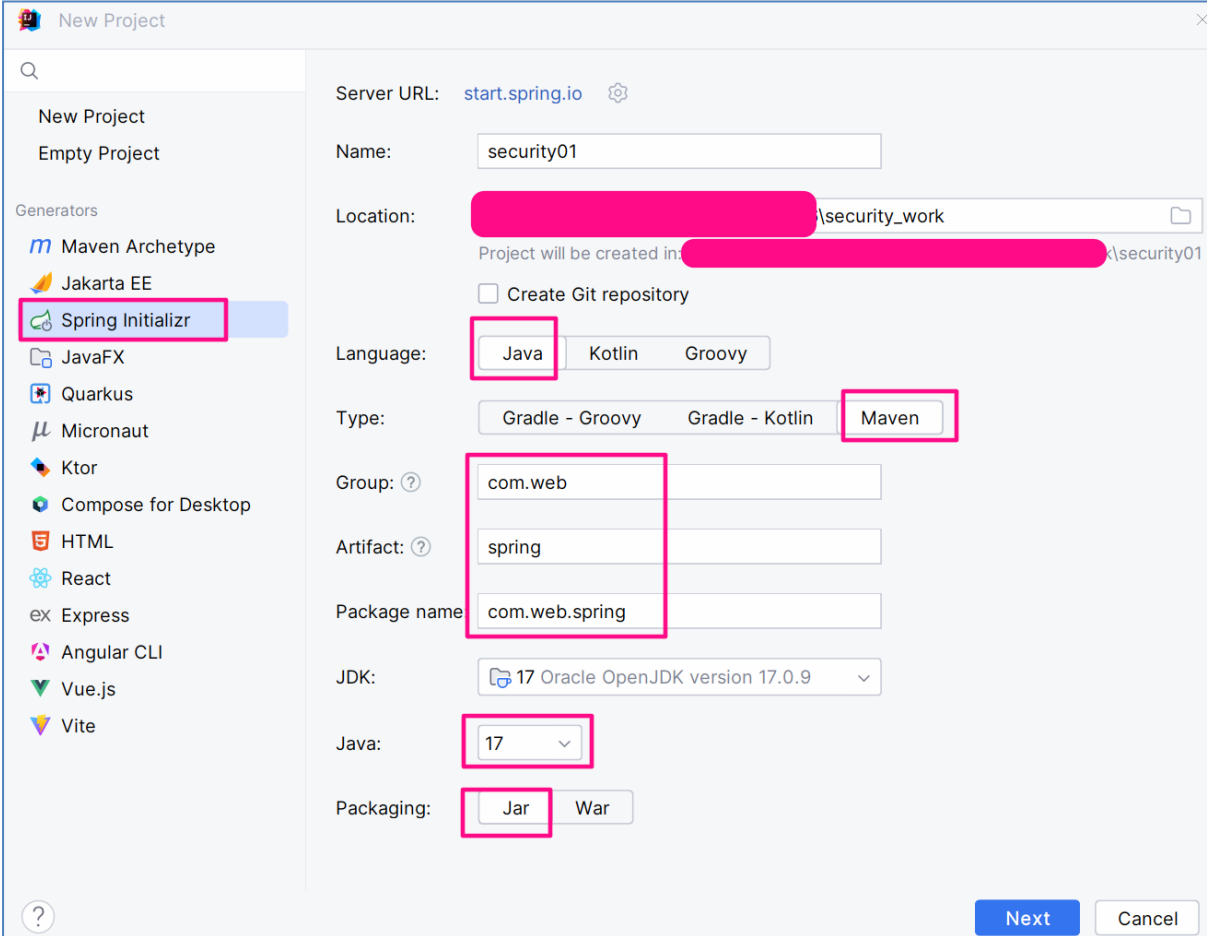
- SpringBoot 3.X버전
- JDK 17버전
- Maven기반
- Spring Security, Spring Web, DevTool,Lombok의존성 주입
- IntelliJ IDEA

Project 1 – security01

Group ID :com.web

Artifact ID : spring

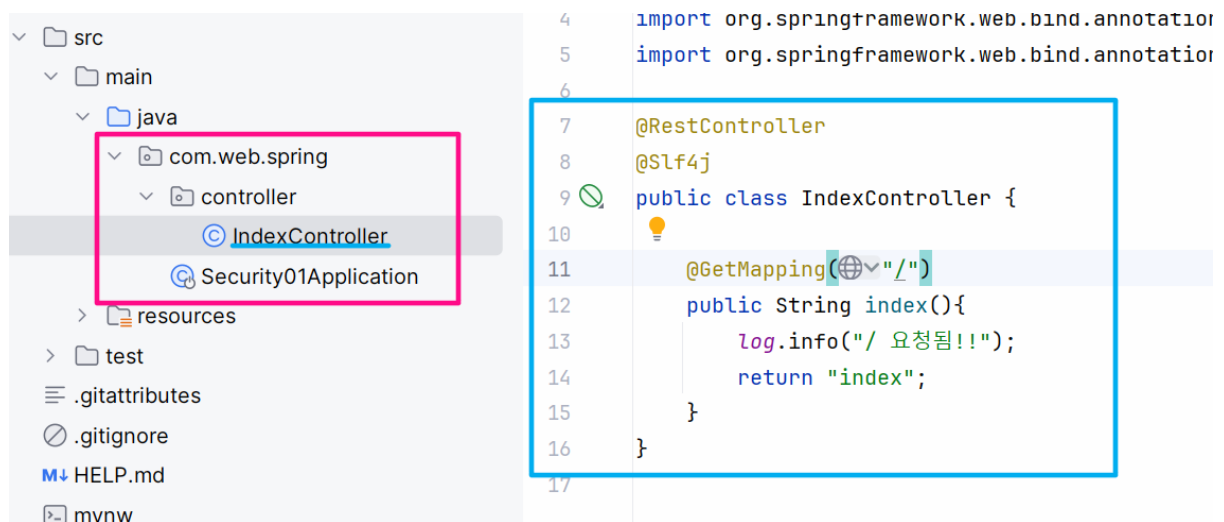
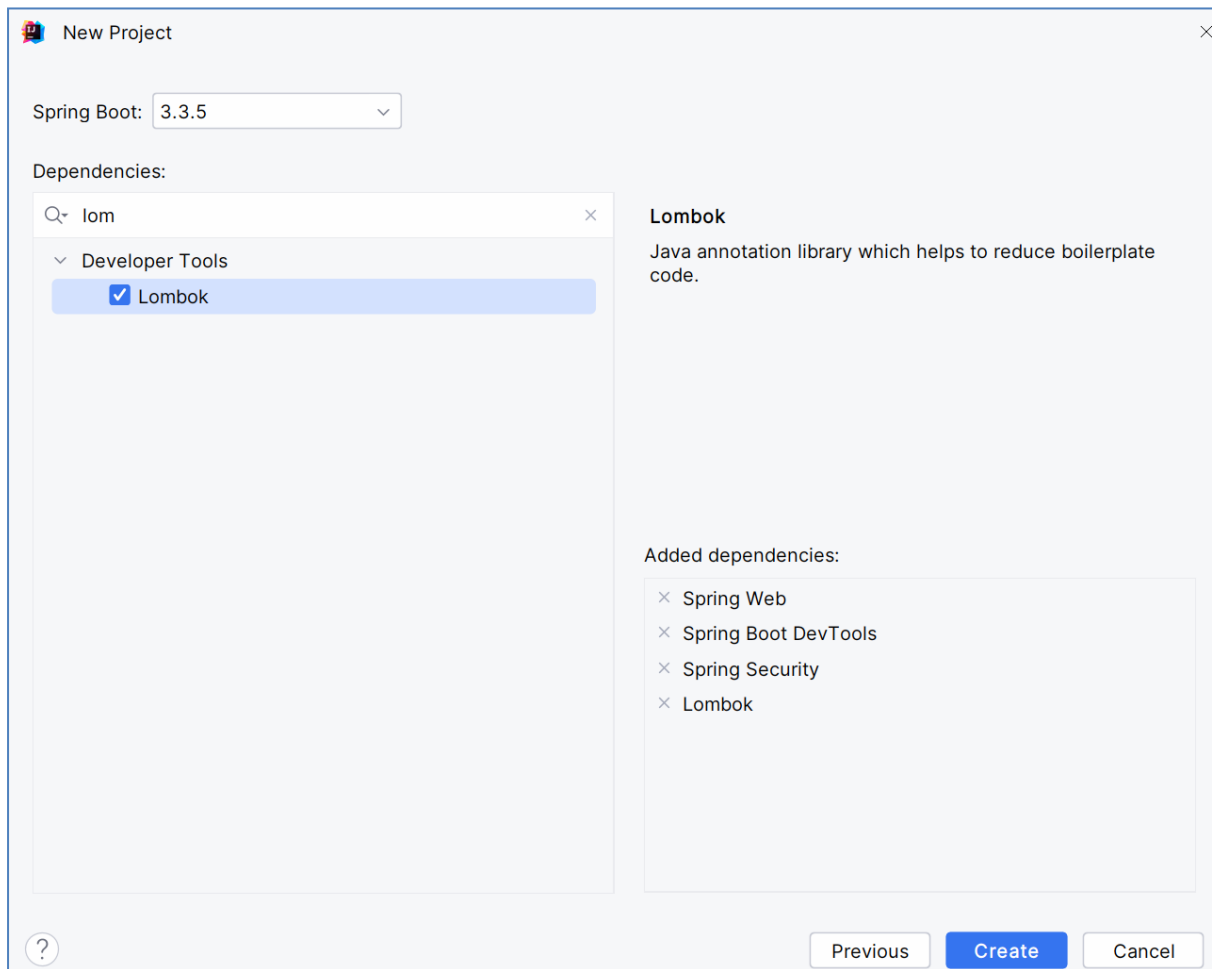
Dependency : Dev Tool, Spring Security, Spring Web,lombok



The image shows the 'New Project' dialog in the Spring Initializr web application. The left sidebar lists various project generators, with 'Spring Initializr' highlighted. The main form contains the following fields and options:

- Server URL:** start.spring.io
- Name:** security01
- Location:** [redacted] \security_work
- Project will be created in:** [redacted] \security01
- ☐ Create Git repository
- Language:** Java (selected), Kotlin, Groovy
- Type:** Gradle - Groovy, Gradle - Kotlin, Maven (selected)
- Group:** com.web
- Artifact:** spring
- Package name:** com.web.spring
- JDK:** 17 Oracle OpenJDK version 17.0.9
- Java:** 17
- Packaging:** Jar (selected), War

At the bottom right, there are 'Next' and 'Cancel' buttons.



1. IndexController 생성

```
@RestController
public class IndexController {

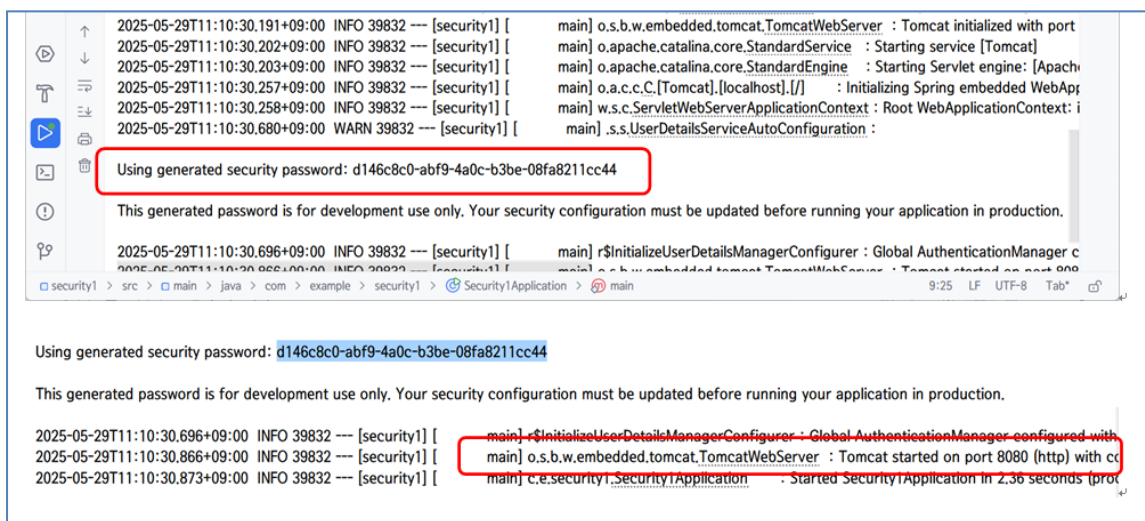
    @GetMapping("/")
    public String index(){
        return "Index";
    }
}
```

1) Maven Dependencies 확인

- spring-security-web 6.3.4
- spring-boot 3.3.5

2) 이 상태에서 서버 가동해서 확인

- tomcat 10버전 8080 가동
- generated security password: aef463dd-00cf-43dc-8de6-27427fcfbf2c
- Global AuthenticationManager configured with UserDetailsService bean with name inMemoryUserDetailsManager

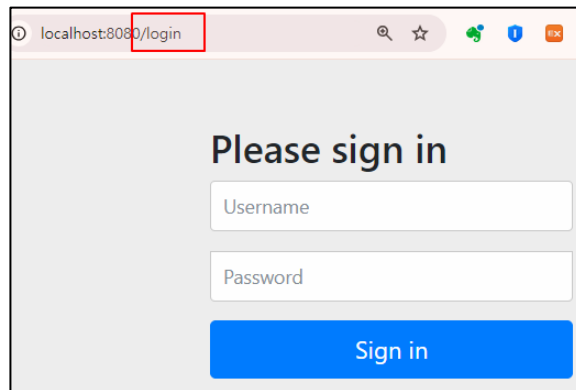


```
your application in production.

gurer : Global AuthenticationManager configured with UserDetailsService bean with name inMemoryUserDetailsManager
       : LiveReload server is running on port 35729
rver   : Tomcat started on port 8080 (http) with context path '/'
n      : Started Security01Application in 2.332 seconds (process running for 3.176)
```

3) 서버가동후 브라우저로 요청 확인

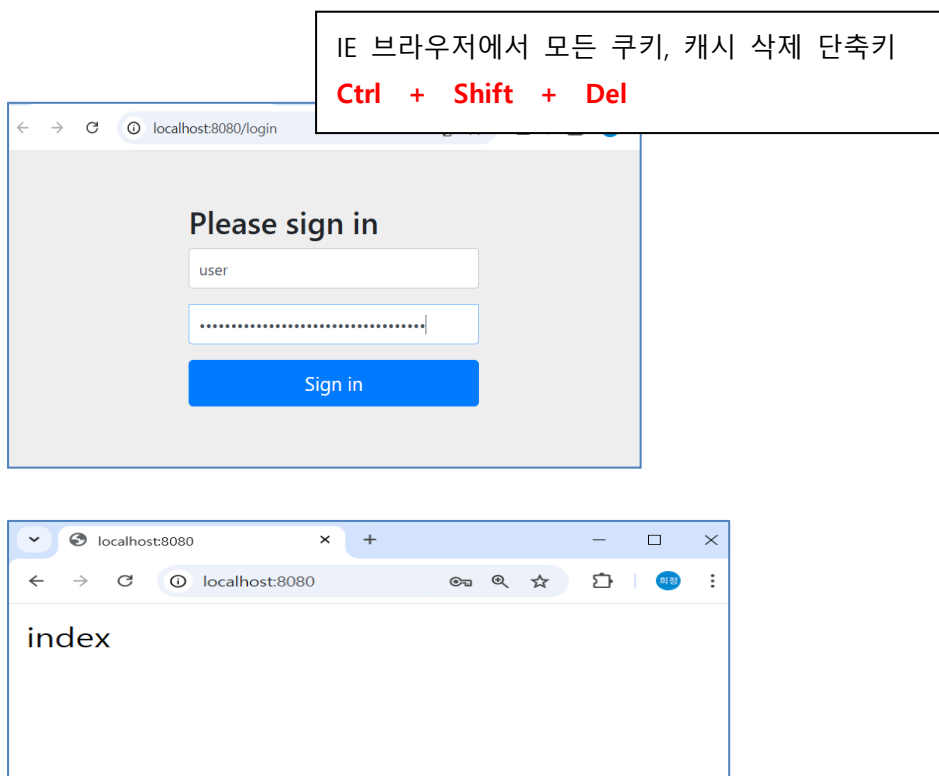
- http://localhost:8080/ --> index 문자 출력이 안되고 로그인 폼이 나온다.
스프링 시큐리티가 작동되면 기본적으로 웹 보안설정에 의해서 로그인 폼이 나온다.



- 폼에 아이디 패스워드를 직접 입력해본다

아이디는 user / 비번은 콘솔에 출력된 문자열을 복사해서 넣는다.

--> 인증 성공하면 index text가 출력된다.



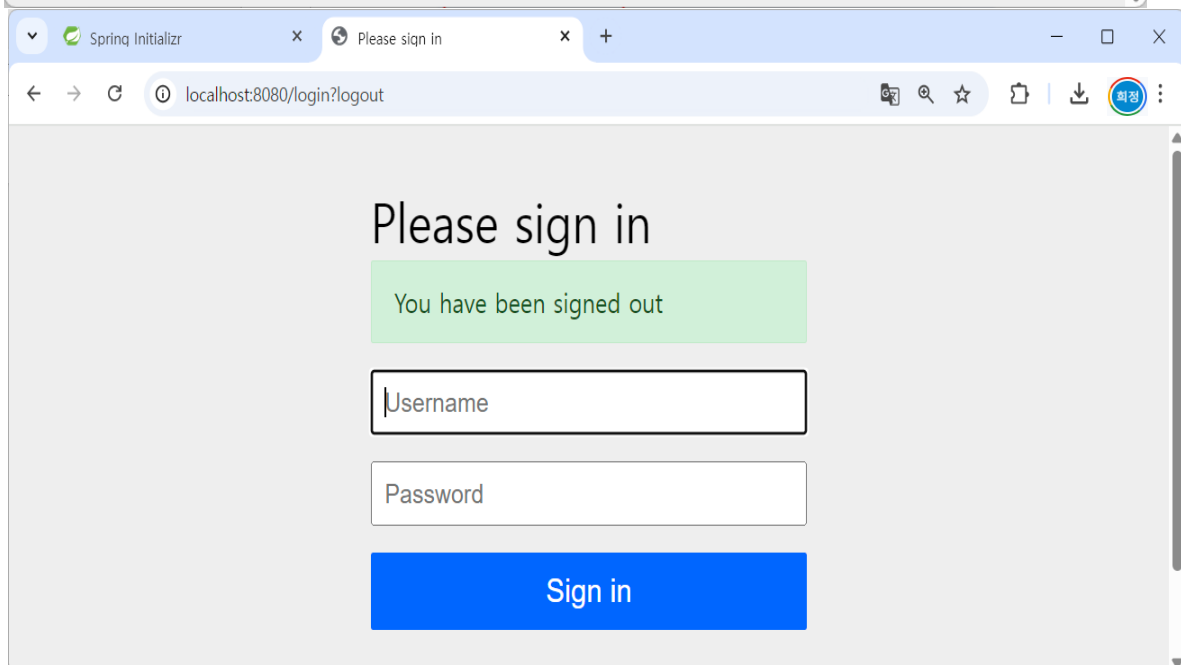
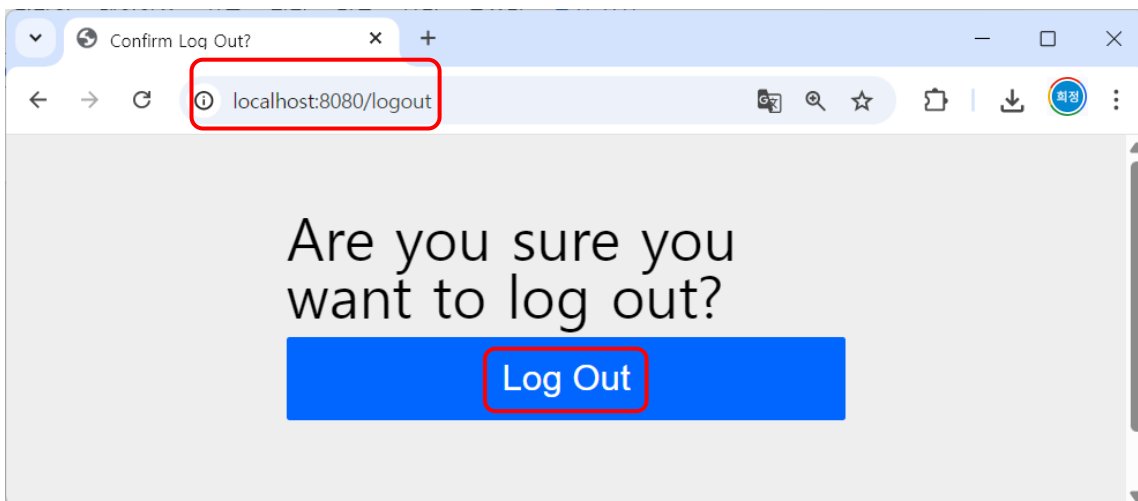
➔ 스프링 Security 의존성을 추가하면 하나의 계정이 추가된다.

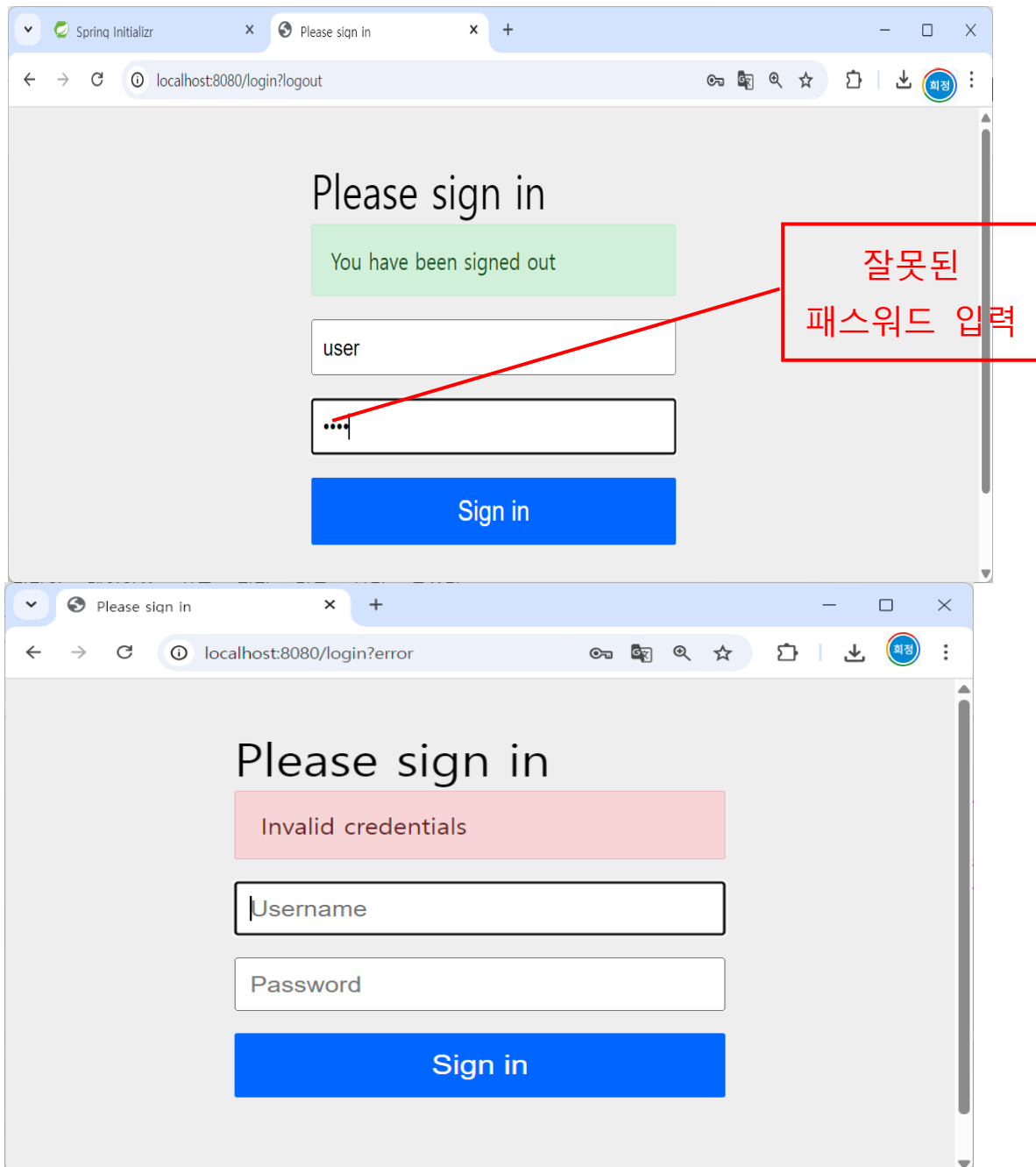
그리고 기본적인 이런 작업이 자동으로 진행된다.

로그인 폼 / 아이디는 user / 패스워드는 랜덤한 문자열 생성

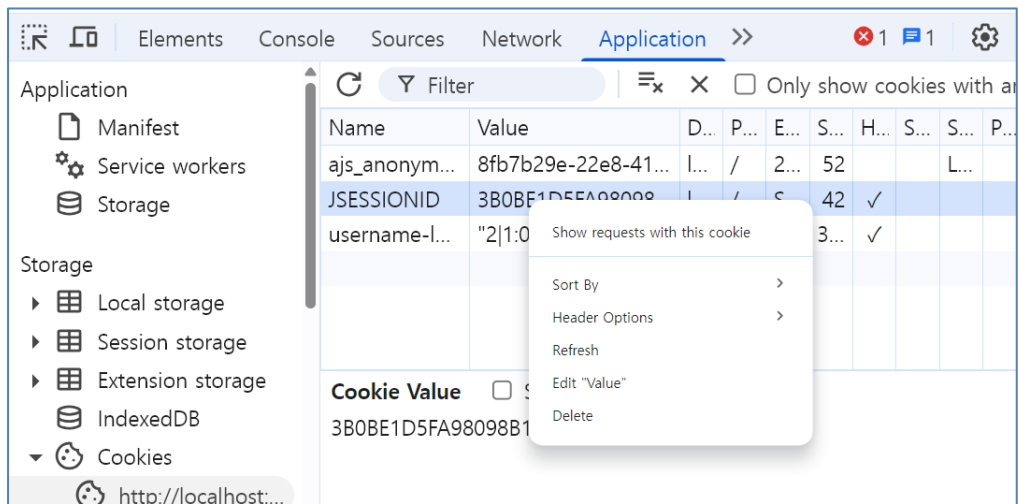
스프링Security가 작동해서 내부적으로 웹 보안 기능이 작동한것이다.

 				
Path	Url	Status	Type	Initiator
 /	http:/...	302	docume...	Other
 /login	http:/...	200	document	:8080/

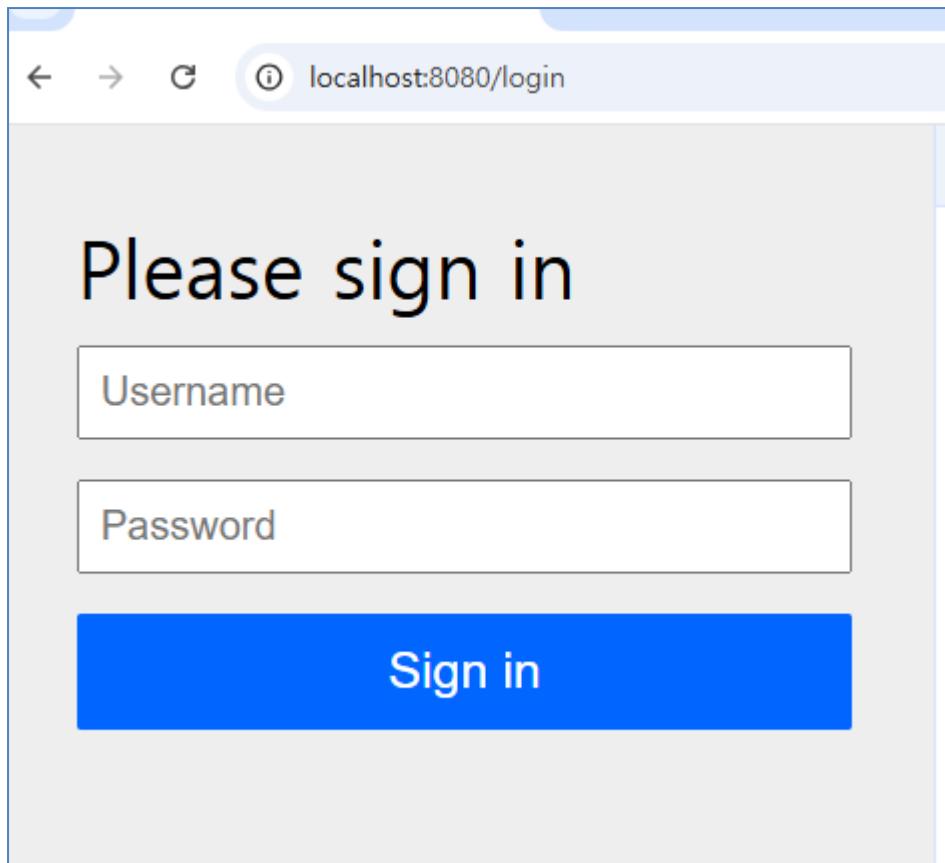




로그인이 성공한 상태에서 Cookies에 있는 JSESSIONID를 삭제하고 <http://localhost:8080/> 요청해본다.



로그아웃이 되어 로그인 폼이 출력된다.



인증 폼에서 마우스 우클릭> 소스보기를 확인한다.

```

<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
<meta name="description" content="">
<meta name="author" content="">
<title>Please sign in</title>
<link href="https://maxcdn.bootstrapcdn.com/bootstrap/4.0.0-beta/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-Y6pD6FV/Vv2HJnA6"
<link href="https://getbootstrap.com/docs/4.0/examples/signin/signin.css" rel="stylesheet" integrity="sha384-oOE/3mOLUMPub4kaC09mrdEhlc+e3ex"
</head>
<body>
<div class="container">
<form class="form-signin" method="post" action="/login">
<h2 class="form-signin-heading">Please sign in</h2>
<p>
<label for="username" class="sr-only">Username</label>
<input type="text" id="username" name="username" class="form-control" placeholder="Username" required autofocus>
</p>
<p>
<label for="password" class="sr-only">Password</label>
<input type="password" id="password" name="password" class="form-control" placeholder="Password" required>
</p>
<input name="_csrf" type="hidden" value="nX7dsriUMCBiKR-TFwONwddawJHxpoGqCxtEBlEw_pR3vR0l_Qzs14nyAkFPHS2glyA5-b147fPAxeuHbi17MGChzqxAiHs-" />
<button class="btn btn-lg btn-primary btn-block" type="submit">Sign in</button>
</form>
</div>
</body></html>

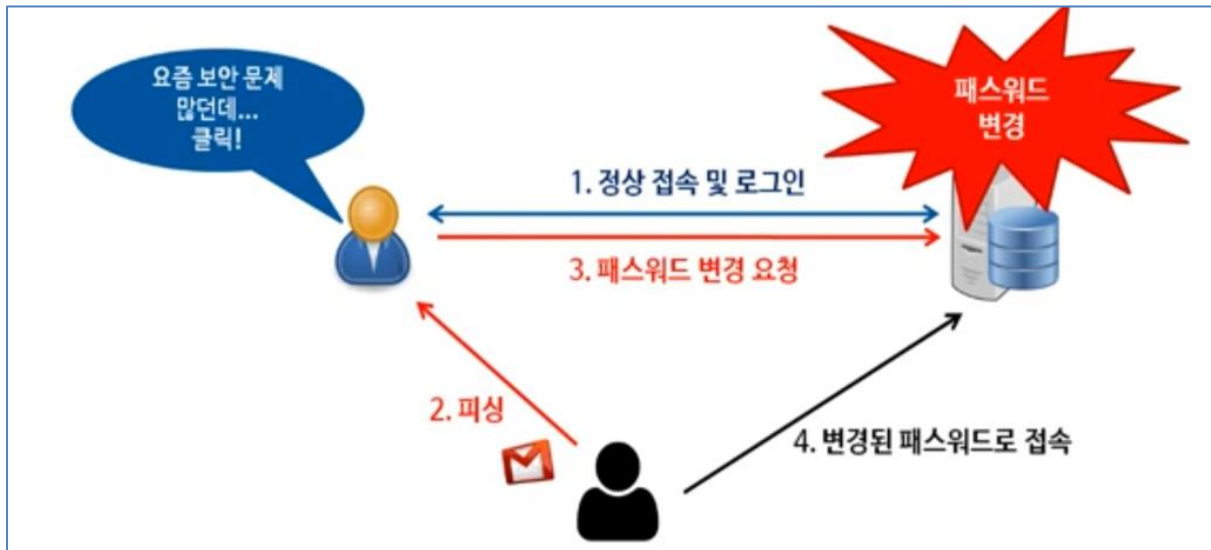
```

CSRF(Cross-Site Request Forgery) 토큰은 웹 애플리케이션에서 CSRF 공격을 방지하기 위해 사용되는 보안 메커니즘이다. CSRF 공격은 사용자가 의도하지 않은 요청을 공격자가 유도하여 특정 웹 애플리케이션에서 악의적인 작업을 수행하게 하는 공격 방식이다. 예를 들어, 로그인된 사용자가 공격자의 악성 링크를 클릭하면, 그 사용자(로그인된 사용자)의 권한으로 웹 애플리케이션에서 원하지 않는 작업(예: 계좌 이체, 비밀번호 변경 등)이 수행될 수 있다.

CSRF 토큰의 원리

CSRF 토큰은 사용자가 웹 애플리케이션에서 보낼 요청에 대한 인증 정보를 포함한 고유한 문자열이다. 이 토큰은 서버가 생성하고 클라이언트에게 전달되며, 클라이언트는 요청을 보낼 때마다 이 토큰을 포함해야 한다. 서버는 요청에 포함된 CSRF 토큰을 확인하여, 요청이 실제로 사용자가 보낸 유효한 요청인지 확인한다.

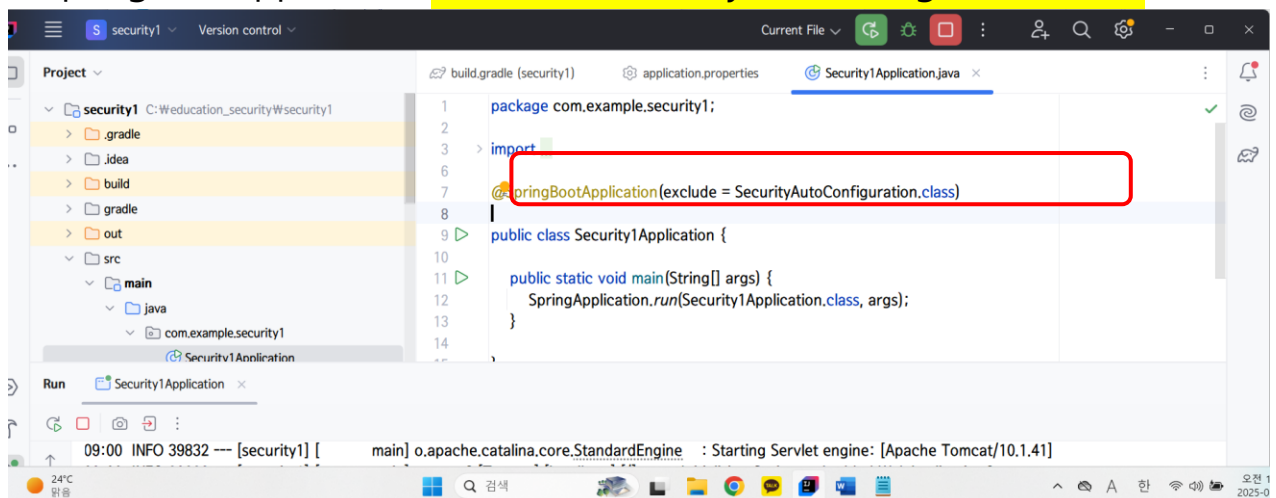
CSRF 토큰은 사용자가 요청을 보낼 때마다 서버와 클라이언트 간의 신뢰할 수 있는 상호작용을 보장하는 중요한 보안 수단으로 이를 통해 악의적인 사이트에서 사용자를 속여 웹 애플리케이션에서 권한 있는 작업을 수행하는 것을 방지할 수 있다.

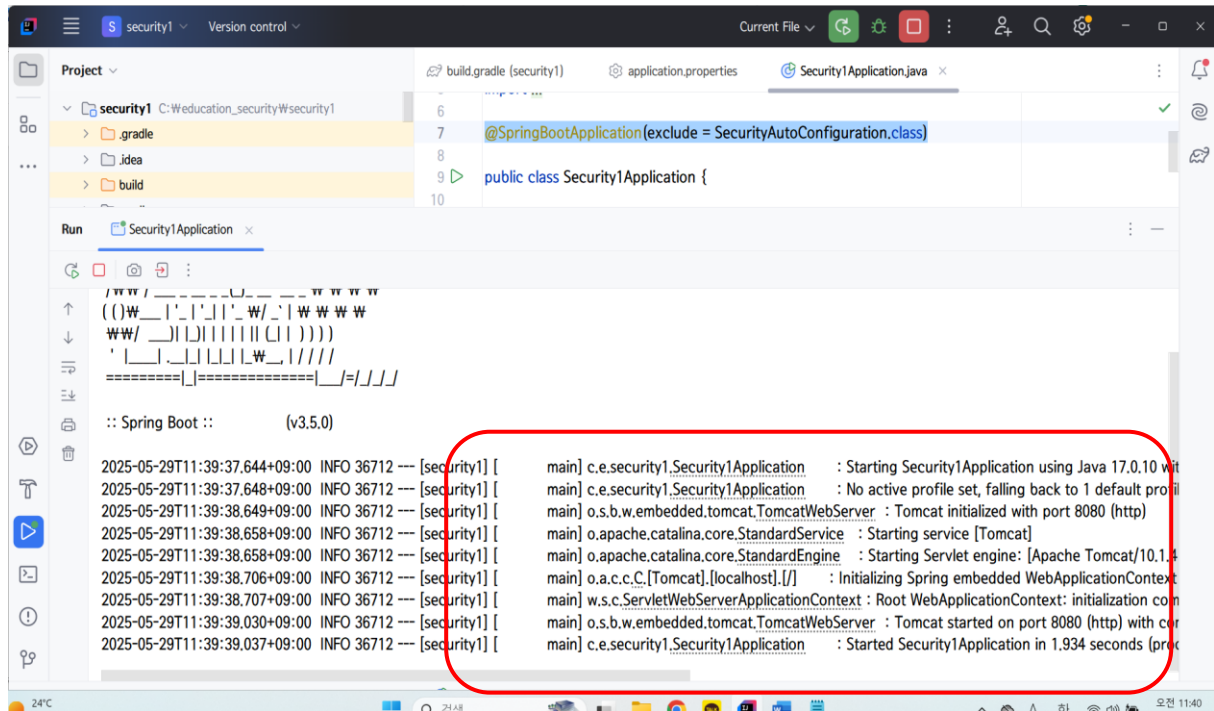


피싱(Phishing)은 사이버 공격의 일종으로, 공격자가 사용자로 하여금 민감한 정보를 제공하게 유도하는 방법이다. 주로 이메일, 가짜 웹사이트, 메시지 등을 통해 사용자를 속여서 **로그인 자격 증명, 신용카드 정보, 사회보장번호, 은행 계좌 정보** 등의 개인정보를 탈취하려고 한다. "피싱"이라는 단어는 "낚시(fishing)"에서 유래했는데, 공격자가 마치 "미끼"를 던져서 사용자가 그것을 물게끔 유도하는 방식에서 이름이 유래한 것이다.

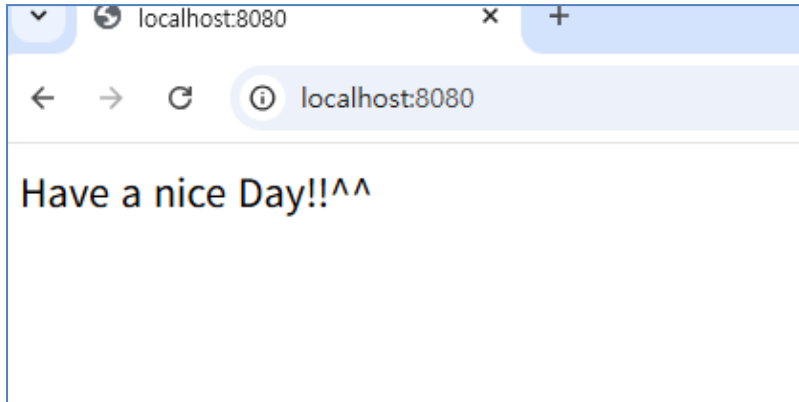
만약, 프로젝트에서 **Seucrity**적용을 빼고 싶다면 아래와 같이 설정한다.

`@SpringBootApplication(exclude = SecurityAutoConfiguration.class)`





<http://localhost:8080/> 요청하면



SecurityConfig 작성

- SpringSecurity 인증과 인가에 대한 설정을 하는 파일
- http요청에 대한 인증과 인가에 대한 설정을 하는 파일
- 모든 코드는 람다형식으로 작성(이유는 Security 6 버전부터는 람다형식만 지원할 예정)

```

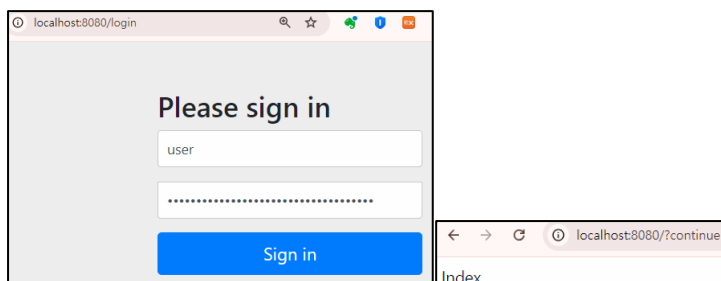
@EnableWebSecurity
@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception{
        //인가정책을 설정
        http
            .authorizeHttpRequests(auth->auth.anyRequest().authenticated()) //어떤 요청에도 인증받겠다
            .formLogin(Customizer.withDefaults()); //폼 로그인 방식으로 인증 받겠다..
        return http.build();
    }
}

```

➔ 서버 다시 재가동 후 실행 - 기본방식이기에 패스워드 자동생성된다.

- 1) <http://localhost:8080/> 요청하면 이전과 동일한 결과



- ➔ 어떤 요청에도 인증 받지 못하면 안되기 때문에 무조건 login페이지로 간다.
➔ 인증 기본 방식은 LoginForm방식이기 때문에 로그인 폼이 뜬다.

2. 사용자정의 계정을 직접 추가

properties파일 | 자바 설정 클래스 2가지 방법이 가능하다.

1) application.properties

3. SERVER RESTART!

```

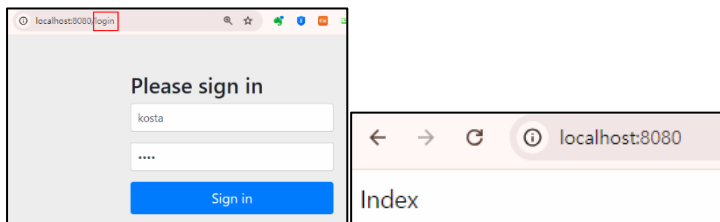
spring.security.user.name=kosta
spring.security.user.password=1234
spring.security.user.roles=USER

```

서버 재가동 되었을 때 살펴보면 패스워드가 자동으로 생성되지 않는다!!

다시 브라우저로 요청해서 직접 등록한 계정으로 로그인 진행한다.

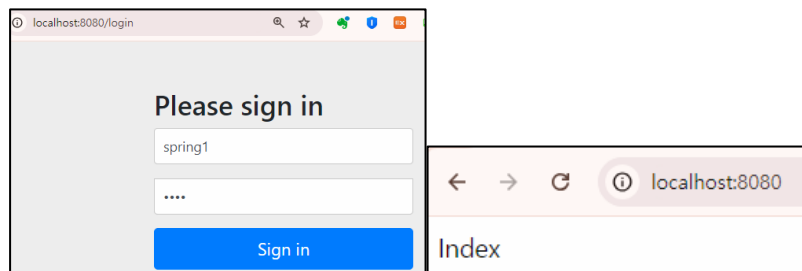
이때 기존 방식대로 login으로 요청해야 한다. (아이디 kosta / 패스워드 1234)



2) 자바 설정 클래스

```
//### 4.여러개 user객체 지정할수 있다.
// 설정파일과 자바빈 2개를 동시에 작업했을때 이 부분이 우선이 된다.
@Bean
public UserDetailsService userDetailsService() {
    UserDetails user=User.withUsername("spring")
        .password("{noop}1234")
        .roles("USER").build();
    //UserDetails 객체는 여러개 가능하다.
    UserDetails user2=User.withUsername("spring1")
        .password("{noop}1234")
        .roles("USER").build();
    UserDetails user3=User.withUsername("spring2")
        .password("{noop}1234")
        .roles("USER").build();
    return new InMemoryUserDetailsManager(user,user2,user3);
}
```

- ➔ 서버 재가동해서 브라우저로 다시 요청
- ➔ 하지만 properties파일에 지정된 kosta / 1234로는 인증 실패한다.
- ➔ 이렇게 사용자 지정방식을 하면 이 또한 InMemory형식으로 메모리에 저장된다.



Spring Security 는 기본적으로 비밀번호를 안전하게 저장하기 위해 해시 함수를 사용한 암호화를 권장한다. 예를 들어, bcrypt, pbkdf2, scrypt 등 다양한 알고리즘을 사용할 수 있다. 그러나 때로는 암호화되지 않은 평문 비밀번호를 처리해야 할 때가 있는데, 이때 {noop}를 사용하면 Spring Security 가 비밀번호를 암호화 없이 그대로 사용하게 된다.

Spring Security 에서 {noop}는 비밀번호 암호화 방식으로 "No Operation" (즉, 암호화하지 않음)을 지정하는 것이다. 이 설정은 주로 **평문(plain text)** 비밀번호를 그대로 사용하는 경우에 사용된다.

Project 2 – security02_LoginForm**Group ID :com.web****Artifact ID : spring****Dependency : Dev Tool, Spring Security, Spring Web, Lombok**

: security01 프로젝트를 복사해서 rename 후 수정사항을 수정한다.

- application.properties파일 추가한 name, password, roles 삭제한다.
- SecurityConfig 파일을 아래와 같이 수정한다.

1. SecurityConfig 작성

```

@EnableWebSecurity
@Configuration
public class SecurityConfig {
    ///### 1. FormLogin으로 정의
    //이거하고 IndexController에서 요청받는 부분을 만들도록 한다.이전 프로젝트 굼어서 작성.
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception{
        http
            .authorizeHttpRequests(auth->auth.anyRequest().authenticated()) //어떤 요청에도 인증받겠다
            .formLogin(Customizer.withDefaults()); //폼 로그인 방식으로 인증 받겠다..
        return http.build();
    }
    @Bean
    public UserDetailsService userDetailsService() {
        UserDetails user=User.withUsername("kosta")
            .password("{noop}1234")
            .roles("USER").build();

        return new InMemoryUserDetailsManager(user);
    }
}

```

1) Customizer → 소스보기(마우스 왼쪽 클릭)

Functional Interface로 customize(T t)함수가 들어있다. T로 내가 지정하는 클래스를 입력
굳이 Customizing 할 필요가 없다면 withDefaluts() 사용한다.

우리는 Customizing 할거니깐 customize(T t) 함수 사용할것이다.

2) Customizer 대신에 form

```

.formLogin(form->form
    //스프링에서 제공하는 로그인폼을 사용할수 없다(확인한후).나중에 로그인폼 만들때 사용하고 좀있다 주석처리
    .loginPage("/loginPage")
    .loginProcessingUrl("/loginProc")
    .defaultSuccessUrl("/", true)//true는 항상 사용, 디폴트는 false
    .failureUrl("/failed")
    .usernameParameter("userId")
    .passwordParameter("userPass")

    .successHandler((request,response,authentication)-> {
        System.out.println("authentication : "+authentication);
        response.sendRedirect("/home");
    })
    .failureHandler((request,response,exception)-> {
        System.out.println("exception : "+exception.getMessage());
        response.sendRedirect("/login");
    })
    .permitAll()
);
return http.build();

```

2. 2)번 그대로 일단 실행하기위해서IndexController 작성

```

@RestController
public class IndexController {

    @GetMapping("/")
    public String index() {
        return "INDEX";
    }
}

```

```

##### 2. loginPage요청과 함께 home요청을 추가
@GetMapping("/loginPage")
public String loginPage() {
    return "loginPage";
}
@GetMapping("/home")
public String home() {
    return "home";
}

```

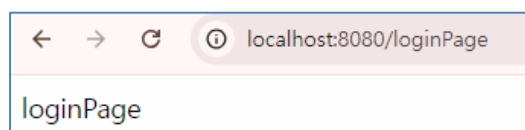
1)서버 가동해서 실행

<http://localhost:8080/> 하면 인증받지 못하면 로그인 폼이 뜬다. 설정 때문에 loginPage출력
→.loginPage 주석처리 안하고 요청하면 바로 loginPage가 뜬다.

```

//인증받지 못하면 로그인 페이지로 간다.
.formLogin(form->form
    //스프링에서 제공하는 로그인폼을 사용할수 없다(확인한후).나중에 로그인폼 만들때 사용하고 좀있다 주석처리
    .loginPage("/loginPage")

```



→이번에는주석처리하고 다시 실행 http://localhost:8080/login

```
.formLogin(form->form
```

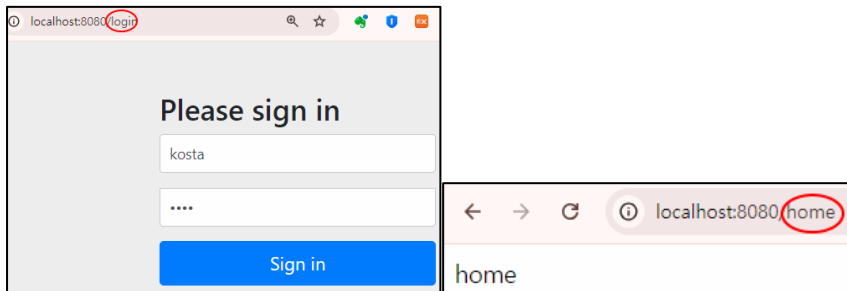
```
//스프링에서 제공하는 로그인폼을
```

```
//loginPage("/loginPage")
```

주석 처리하고 다시 실행

→인증폼이 나타난다

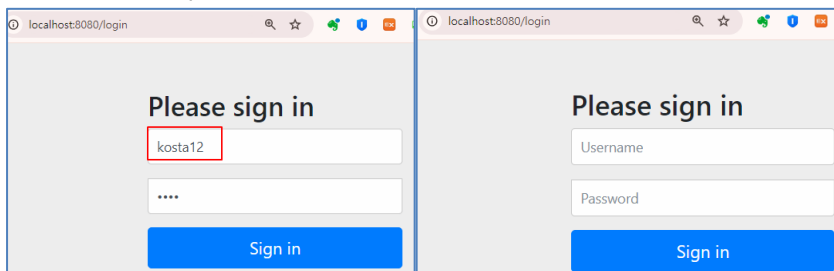
→kosta / 1234 입력



→로그인 폼이 뜨면 개발자도구에서 폼 이름을 직접 확인한다!!

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
    <meta name="description" content="">
    <meta name="author" content="">
    <title>Please sign in</title>
    <link href="https://maxcdn.bootstrapcdn.com/bootstrap/4.0.0-beta/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-Y6p06FV/Vv2HJnA6"
    <link href="https://getbootstrap.com/docs/4.0/examples/signin/signin.css" rel="stylesheet" integrity="sha384-o0E/3m0LUMPub4kaC09mrdEhlc+e3ex"
  </head>
  <body>
    <div class="container">
      <form class="form-signin" method="post" action="/loginProc">
        <h2 class="form-signin-heading">Please sign in</h2>
        <p>
          <label for="username" class="sr-only">Username</label>
          <input type="text" id="username" name="userId" class="form-control" placeholder="Username" required autofocus>
        </p>
        <p>
          <label for="password" class="sr-only">Password</label>
          <input type="password" id="password" name="userPass" class="form-control" placeholder="Password" required>
        </p>
        <input name="_csrf" type="hidden" value="fab6_JHYIK089-q8KC1dp800SErQStestMIMuLLIRQLrX4YgR5P0mqjrpJSRIN-FSQBpnSqAZXKyeGBW_dv3oeDfDCIPecT" />
        <button class="btn btn-lg btn-primary btn-block" type="submit">Sign in</button>
      </form>
    </div>
  </body></html>
```

→이번에는 id, password를 다르게 입력해서 어떤 페이지로 연결되는지 확인

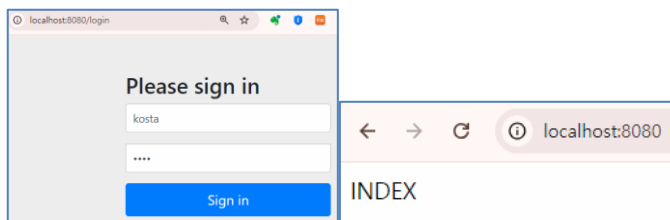


→successHandler, failureHandler 부분 주석처리하고 로그인 요청한다.

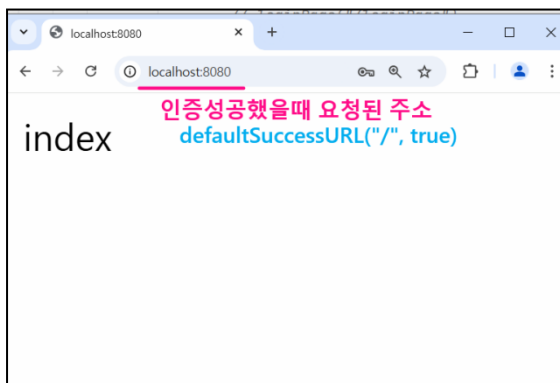
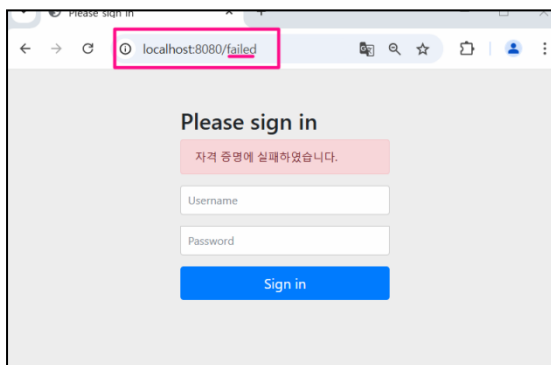
```

/*
    마지막으로 이 부분 주석처리하고 로그인 요청해보자
    .successHandler((request,response,authentication)-> {
        System.out.println("authentication : "+authentication);
        response.sendRedirect("/home");    //성공하면 home으로
    })
    .failureHandler((request,response,exception)-> {
        System.out.println("exception : "+exception.getMessage());
        response.sendRedirect("/login");    //실패하면 login으로
    })
*/
.permitAll()

```



→ 사실 successHandler, failureHandler를 정의했다면 위에 defaultSuccessUrl, failureUrl은 의미없는 기능이다.



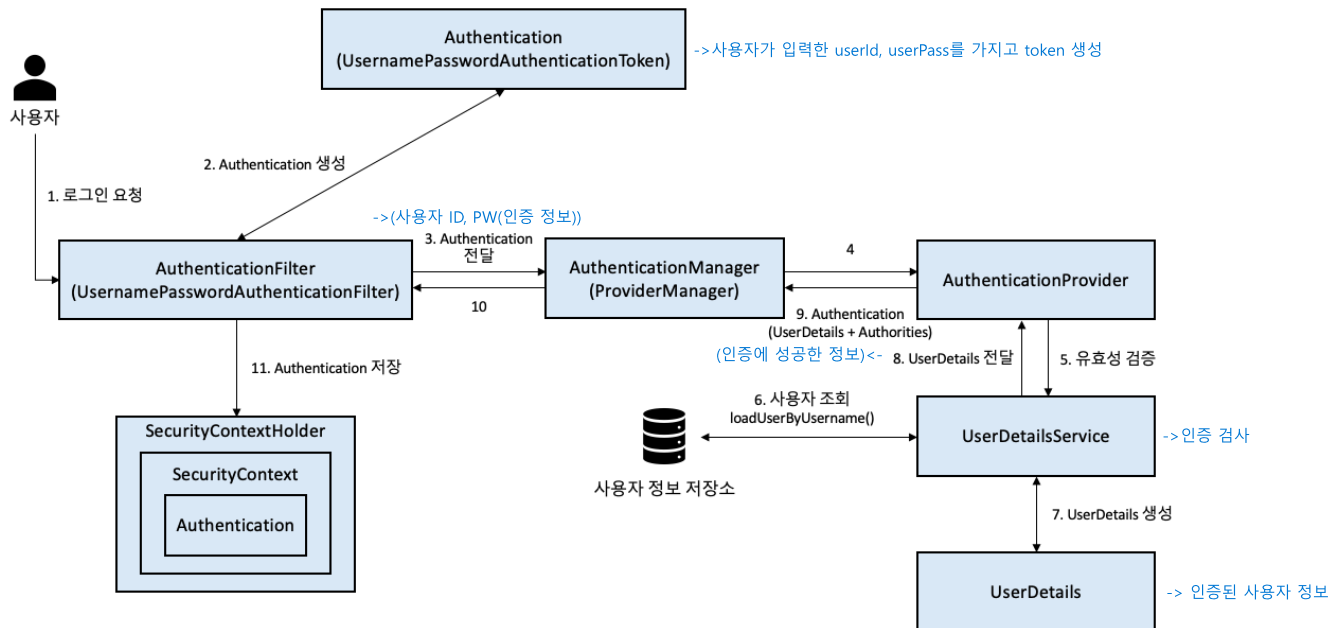
```

MySecurityConfig.java x Security3Application.java
7 import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
8 import org.springframework.security.web.SecurityFilterChain;
9
10 @Configuration
11 @EnableWebSecurity
12 public class MySecurityConfig {
13     @Bean
14     @public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
15         http.authorizeHttpRequests(requests -> requests
16             .requestMatchers("/images/**", "/*.html").permitAll()
17             .anyRequest().authenticated() )
18         .formLogin(Customizer.withDefaults());
19         SecurityFilterChain chain = http.build();
20         chain.getFilters().forEach(System.out::println);
21         return chain;
22     }
23 }

```

org.springframework.security.web.session.DisableEncodeUrlFilter@6fff46bf
 org.springframework.security.web.context.request.async.WebAsyncManagerIntegrationFilter@1835dc92
 org.springframework.security.web.context.SecurityContextHolderFilter@45c9b3
 org.springframework.security.web.header.HeaderWriterFilter@3a6045c6
 org.springframework.security.web.csrf.CsrfFilter@3234474
 org.springframework.security.web.authentication.logout.LogoutFilter@25f15f50
 org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter@1d8b0500
 DefaultResourcesFilter [matcher=Ant [pattern='/default-ui.css', GET], resource=org/springframework/security/default-ui.css]
 org.springframework.security.web.authentication.ui.DefaultLoginPageGeneratingFilter@d3f4505
 org.springframework.security.web.authentication.ui.DefaultLogoutPageGeneratingFilter@3aaa3c39
 org.springframework.security.web.savedrequest.RequestCacheAwareFilter@6680f714
 org.springframework.security.web.servletapi.SecurityContextHolderAwareRequestFilter@5a936e64
 org.springframework.security.web.authentication AnonymousAuthenticationFilter@65a9ea3c
 org.springframework.security.web.access.ExceptionTranslationFilter@63300c4b
 org.springframework.security.web.access.intercept.AuthorizationFilter@748a654a

3. Spring Security 로그인 인증 처리과정(JWT 사용 안 했을 경우)



1. 사용자 요청(username, password 폼으로 입력됨) <http://localhost:8080/login>
2. AuthenticationFilter (UsernamePasswordAuthenticationFilter)
에서 전달 받은 username 과 password 를 가지고 Authentication 을 생성함.
3. 생성된 인증 Authentication 을 인증매니저 (AuthenticationManger)에 전달한다.
4. 인증매니저는 받은 인증을 Provider 에 전달한다. 이때 이 Provider 는 유효성 검증을 하기 위해 UserDetailsService 에 전달한다.
5. UserDetailsService 는 실제 저장 계층(DB)를 확인하여(지금은 Bean 에 저장된 InMemory) 유저가 있는지, 비밀번호가 맞는지 확인하고, 검증이 성공적으로 이루어지게 되면, UserDetails 를 생성한다.
6. 생성된 UserDetails 를 역순으로 다시 전달한다.
7. 최종 Filter 까지 도달하면 인증 정보를 담고 있는 Authentication 을 시큐리티컨텍스트 SecurityContext 에 저장한다. 여기까지 진행되면 로그인 인증 완료!!
8. SecurityContext 에 저장된 유저정보 Authentication 을 Spring Controller 에서 사용할 수 있다..

```
@GetMapping("/home")
public String home(@AuthenticationPrincipal UserDetails userDetails) {
    log.info("/home 요청됨.. ");
    log.info("userDetails :{ } ", userDetails);
}
```

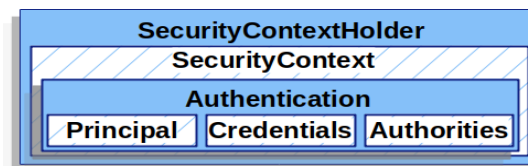
```

Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
UserDetails user = (UserDetails)authentication.getPrincipal();
Collection<? extends GrantedAuthority> authorities = user.getAuthorities();

Iterator<? extends GrantedAuthority> iter = authorities.iterator();
while(iter.hasNext()){
    GrantedAuthority auth = iter.next();
    String role = auth.getAuthority();
    log.info("Authentication role = {} ", role);
}

```

최종적으로 사용자 SecurityContext 에 Authentication 객체로 저장한다는 것은
Spring Security 가 전통적인 세션-쿠키 기반의 인증 방식을 사용한다는 것을 의미한다.



```

18 public class SecurityConfig {
19     @Bean
20     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception{
21         log.info("SecurityFilterChain=====>"); 3)
22         http.authorizeHttpRequests(auth->auth.anyRequest().authenticated())
23             .formLogin(form->form
24                 //loginPage("/loginPage")
25                 .loginProcessingUrl("/loginProc")
26                 .defaultSuccessUrl( defaultSuccessUrl: "/", alwaysUse: true)
27                 .failureUrl( authenticationFailureUrl: "/failed")
28                 .usernameParameter("userId")
29                 .passwordParameter("userPass")
30                 .successHandler((request,response,authentication)->{
31                     System.out.println("authentication = " + authentication);
32                     response.sendRedirect("/home");
33                 })
34                 .failureHandler((request, response, exception) -> {
35                     System.out.println("exception = " + exception.getMessage());
36                     response.sendRedirect("/login");
37                 })
38             .permitAll();
39
40         return http.build();
41     }
42
43
44     @Bean
45     public UserDetailsService userDetailsService(){
46         log.info("UserDetailsService=====>"); 1)
47         UserDetails user= User.withUsername("kosta").password("{noop}1234").roles("USER").build();
48         log.info("UserDetailsService=====>user :: {}",user); 2)
49         return new InMemoryUserDetailsManager(user);
50     }

```

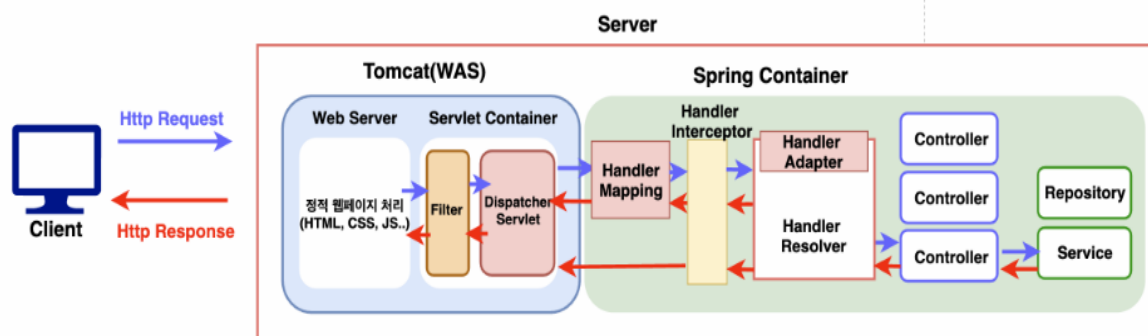
4. 프로젝트 콘솔 출력 결과

```

SecurityConfig...userDetailsService call.. 1
SecurityConfig...userDetailsService call..kosta !! 2
2024-09-13T12:27:21.332+09:00 INFO 6508 --- [Securi
SecurityConfig...securityFilterChain call.. 3
2024-09-13T12:27:21.731+09:00 INFO 6508 --- [Securi
2024-09-13T12:27:21.779+09:00 INFO 6508 --- [Securi
2024-09-13T12:27:21.788+09:00 INFO 6508 --- [Securi
2024-09-13T12:27:31.488+09:00 INFO 6508 --- [Securi
2024-09-13T12:27:31.496+09:00 INFO 6508 --- [Securi
2024-09-13T12:27:31.501+09:00 INFO 6508 --- [Securi
index controller 4

```

5.서블릿 컨테이너와 스프링 컨테이너상에서의 Spring Security Architecture



우리는 Spring MVC Framework에 대한 공부를 이미 마쳤고 위 아키텍처 구조는 익숙하다. Spring MVC의 출발점은 모든 요청을 서버상에서 최초로 받는 Dispatcher Servlet에서 시작된다. 하지만 Java Servlet Filter(DelegationFilterProxy)는 Dispatcher Servlet으로 가는 요청을 WAS 영역에서 가로채고 이 요청을 곧장 Spring Security Filter로 위임하면서 전달한다.

(Servlet Filter와 Security Filter 모두 DispatcherServlet 앞에서 모든 요청을 가로챈다. 위 그림에서 DispatcherServlet 앞에 있는 Filter부분에 해당한다.)

아래에서 조금 더 구체적으로 정리해 보자.

Spring Security란, Http Request가 DispatcherServlet으로 도달하기 전 **Servlet의 Filter**를 기반으로 Request에 대한 인증과 인가를 적용시켜주는 프레임워크이다.

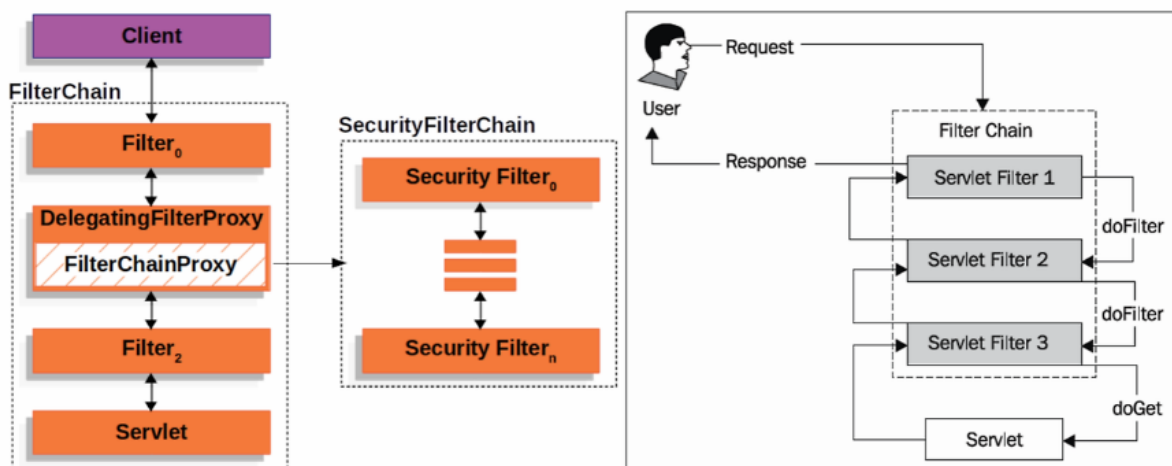
일반적으로 클라이언트에서 서버로 요청을 보내면, DispatcherServlet이 하나의 HttpServletResponse를 받아서 요청을 처리하고 HttpServletResponse 응답을 클라이언트로 보낸다. 그런데, 하나 이상의 **Filter**가 포함된다면, 클라이언트에서 보낸 요청이 Servlet으로 전달되기 전에 **Filter**를 거치게 된다.

클라이언트가 애플리케이션에 하나의 요청을 보내면, 컨테이너는 하나의 **FilterChain**을 생성한다. **FilterChain** 내부의 **Filter**는 말 그대로 '필터'의 역할을 하게 된다. 클라이언트에서 보낸 요청이 다음 **Filter**나 **Servlet**에 전달되지 않도록 걸러낼 수 있다.

GET <http://localhost:8080/memo/123> 을 한다고 생각해보자. 이 API는 인가(권한 체크)를 필요로 한다. 해당 memo의 작성자가 아니면 볼 수 없어야 하기 때문이다.

이런 종류의 인증/인가 기능을 Request가 우리 서버(Servlet)에 도착하기 전에 미리 수행해주는(걸러 주는) 역할을 하는 게 Filter이다.

+이러한 **Filter**들이 묶여 있는 것을 **SecurityFilterChain**이라고 한다.



DelegatingFilterProxy의 내부에는 **FilterChainProxy**라는 것이 있는데, 이것도 Spring Security에서 제공하는 **Filter**이다.

FilterChainProxy는 **DelegatingFilterProxy**를 통해 받은 요청과 응답을 **SecurityFilterChain**에 전달하고 작업을 위임하는 역할을 한다.

SecurityFilterChain은 인증을 처리하는 여러 개의 **SecurityFilter**를 담은 **FilterChain**이다. 또, **FilterChainProxy**를 통해 Servlet Filter와 연결되고 **FilterChainProxy**에서 요청에 대해 호출할 **SecurityFilter**를 결정한다.

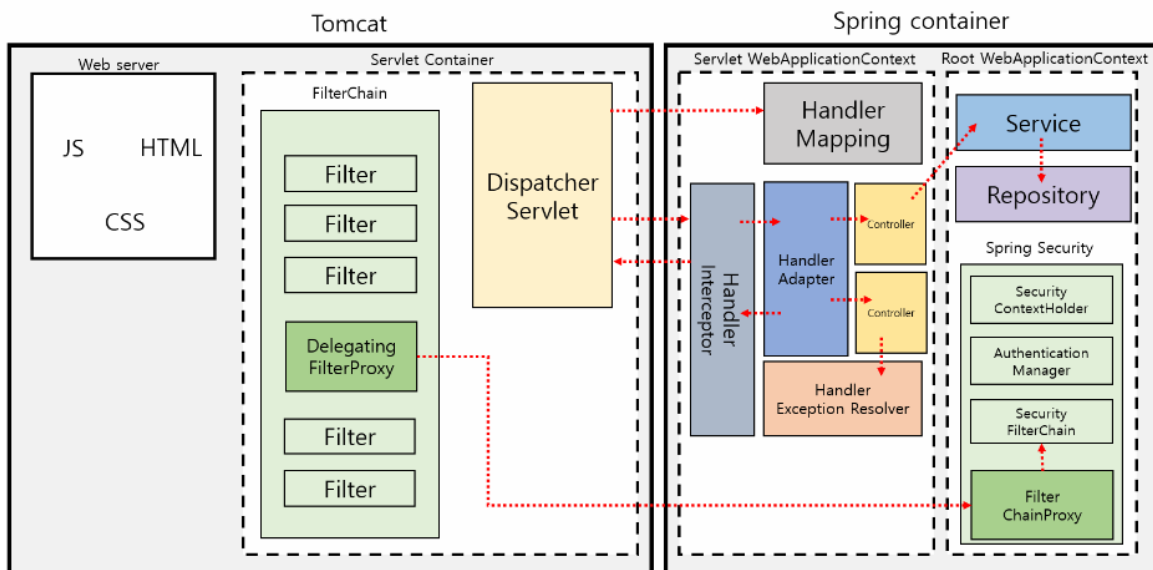
SecurityFilterChain에 담긴 **SecurityFilter**들이 **Filter**로 동작하며 요청이 걸러지게 되는 것이다.

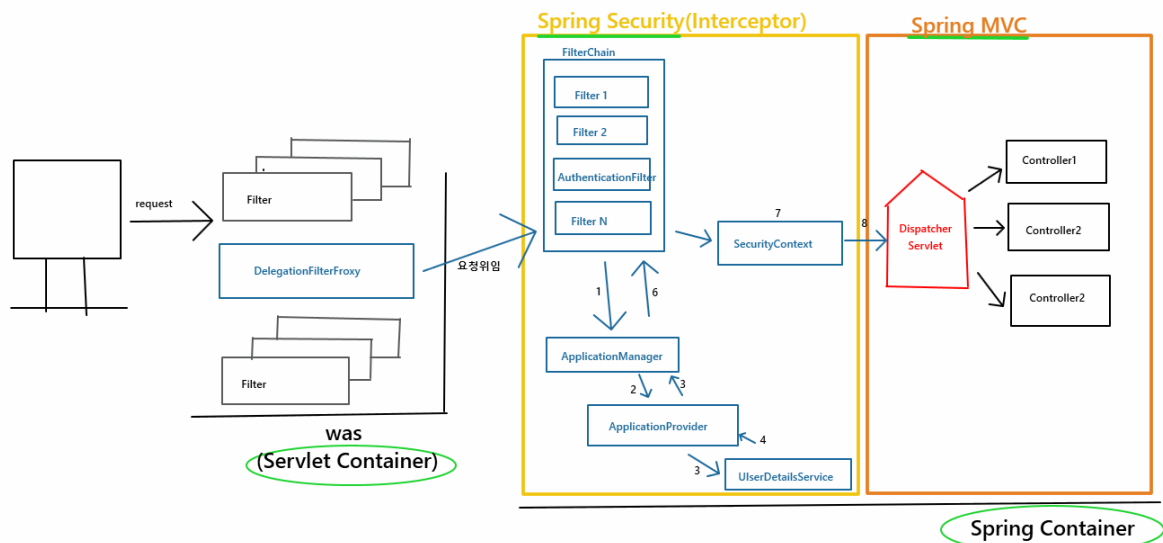
DelegatingFilterProxy에서 바로 **SecurityFilterChain**을 실행시킬 수도 있지만, 중간에 **FilterChainProxy**를 두는 이유는 바로 서블릿을 지원하는 시작점 역할을 하기 위해서다.

만약 서블릿에서 문제가 발생한다면 **FilterChainProxy**의 문제라는 것을 바로 알 수 있다.

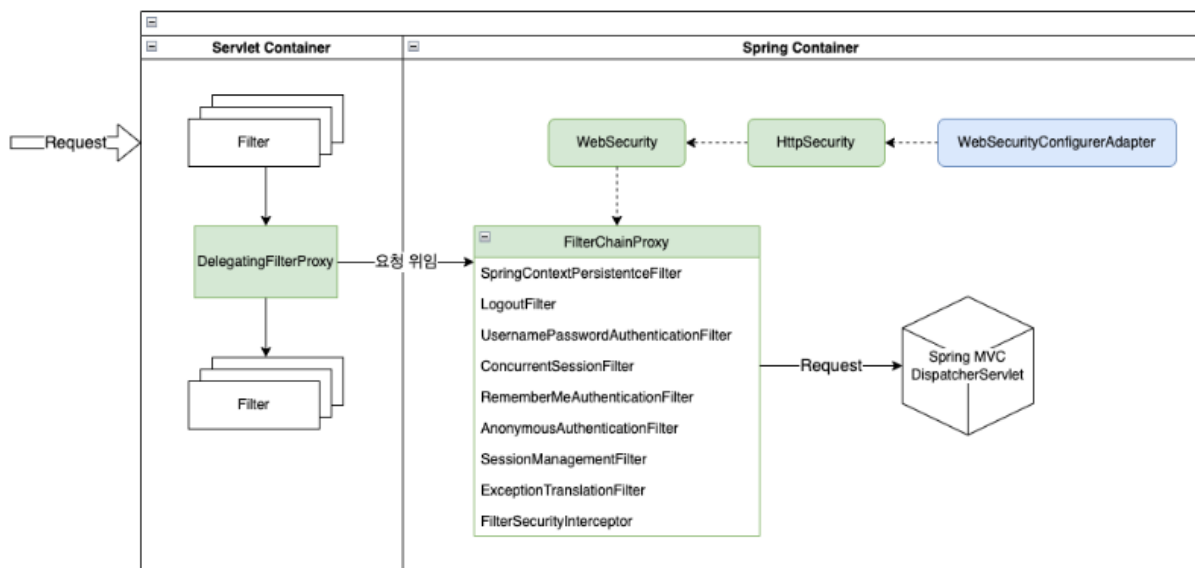
아키텍처를 그림으로 살펴보자

아래 2개의 그림은 결과적으로 같은 그림이다.





결과적으로 Servlet Container의 DelegatingFilterProxy 와
Spring Container의 FilterChainProxy는 클라이언트의 요청에 대해 아래처럼 동작한다.
그리고 이러한 요청이 DispatcherServlet에게 전달된다.

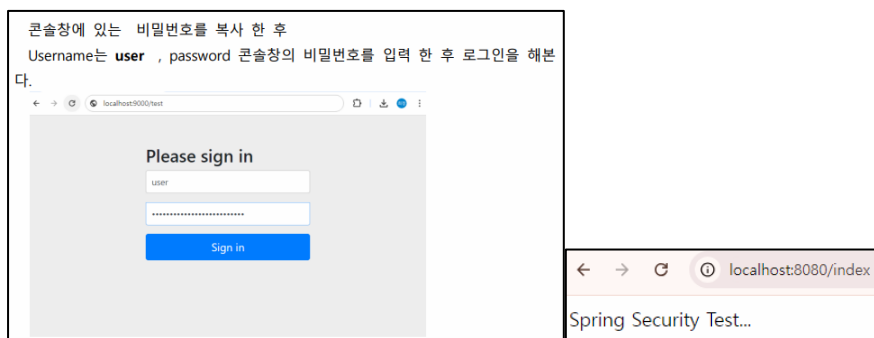


Project 3- security03_Config**Group ID :com.web****Artifact ID : spring****Dependency : Dev Tool, Spring Security, Spring Web,Lombok****IndexController**

```

@RestController
@Slf4j
public class IndexController {
    @GetMapping("/index")
    public String index() {
        Log.info("index 요청함...");
        return "Spring Security Test..";
    }
}

```



boot환경에서는 스프링시큐리티 dependency를 추가하면 자동으로 DelegatingFilterProxy 가 등록되어 시큐리티가 활성화 된다.(스프링부트 3.X 스프링 시큐리티 6.X → 람다형식으로 변경됨)

1. 시큐리티 인가 및 설정을 담당하는 클래스 등록하기 : SecurityConfig.java

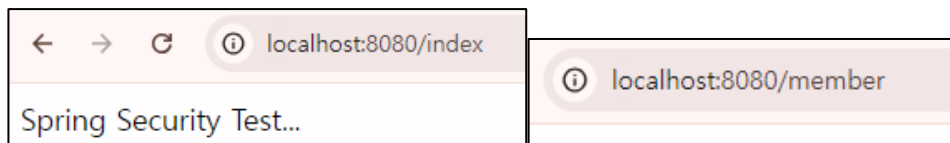
```

@EnableWebSecurity
@Configuration
@Slf4j
public class SecurityConfig {
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception{
        Log.info("===SecurityFilterChain filterChain(HttpSecurity http) call===");
        http.csrf((auth) -> auth.disable()); //csrf공격 방어하기 위해서 토큰 주고받는 부분 비활성화
        //Form 로그인 방식 disable -> React에서 폼으로 연결할 것이기에
        //disable 를 설정하면 시큐리티의 UsernamePasswordAuthenticationFilter비활성됨.
        http.formLogin((auth) -> auth.disable());

        //요청 경로별 인가작업 설정
        http.authorizeHttpRequests((auth)->auth
            .requestMatchers("/index", "/members", "/members/**", "/boards").permitAll()
            .requestMatchers("/admin").hasRole("ADMIN")
            .anyRequest().authenticated());
        return http.build();
    }
}

```

위 파일을 설정한 후 브라우저로 다시 요청한다.



다른이름으로요청시 403뜨다!!

콘솔메시지...

```

Global AuthenticationManager configured with UserDetailsServiceImpl bean with name inMemoryUserDetailsManager
SecurityFilterChain filterChain(HttpSecurity http) call....
LiveReload server is running on port 35729
Tomcat started on port 8080 (http) with context path '/'
Started Test1Application in 2.445 seconds (process running for 3.464)
Initializing Spring DispatcherServlet 'dispatcherServlet'
Initializing Servlet 'dispatcherServlet'
Completed initialization in 7 ms
index 요청함...

```

→Controller가 요청을 받기전 Filter가 그 요청을 가로채서 먼저 수행됨을 알수 있다.

<https://dev-allday.tistory.com/73> 사이트 참조